Дискретная математика

SAT

Константин Чухарев

Осень 2026

Задача SAT

> *Нет царского пути в геометрию.*
>
> ***— Евклид***

Выполнимость

От верификации аппаратуры до головоломок — всё сводится к SAT.

Определение 1 (КНФ-формула)

- **КНФ** — формула $\varphi = C_1 \wedge ... \wedge C_m$, где каждый дизъюнкт — дизъюнкция литералов.
- **Литерал** — переменная x или её отрицание $\overline{x}$.

Определение 2 (SAT)

Существует ли **модель** — выполняющий набор значений?

Формула **выполнима**, если модель существует, иначе — **невыполнима**.

Пример

$\varphi = (x \vee \overline{y}) \wedge (\overline{x} \vee y \vee z) \wedge (\overline{z})$.

Присваивание $x = 1, y = 1, z = 0$ выполняет все дизъюнкты.

Выполнимость и тавтология

Выполнимость — не то же самое, что общезначимость (*validity*).

x выполнима при $x = 1$, но не тавтология.

Тавтология истинна при всех присваиваниях.

Пример

$(x) \wedge (\overline{x})$ — невыполнима: противоречие.

Это простейшая невыполнимая формула.

Почему КНФ?

КНФ — инженерный компромисс.

DNF-SAT тривиальна: дизъюнкция конъюнкций выполнима, если хотя бы один терм не содержит переменную вместе с её отрицанием.

КНФ — достаточно регулярна и достаточно выразительна.

k-SAT

Определение 3

***k*-SAT** — SAT, где каждый дизъюнкт содержит ровно k литералов.

При $k = 2$ задача разрешима за полиномиальное время, при $k = 3$ она NP-полна.

Один лишний литерал — скачок сложности.

Пример

$(x \vee y) \wedge (\overline{x} \vee z)$ — 2-SAT.

$(x \vee y \vee z)$ — 3-SAT.

Алгоритм для 2-SAT

Дизъюнкт $(l_1 \vee l_2)$ равносилен двум импликациям:

$$\overline{l_1} \to l_2, \quad \overline{l_2} \to l_1.$$

Определение 4 (Граф импликаций)

Вершины — литералы. Рёбра $\overline{l_1} \to l_2$ и $\overline{l_2} \to l_1$ для каждого дизъюнкта.

Теорема 1 (Критерий 2-SAT)

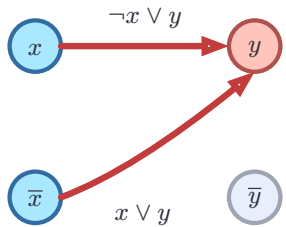
Формула невыполнима тогда и только тогда, когда x и $\overline{x}$ в одной компоненте сильной связности. Если они взаимно достижимы, то x влечёт $\overline{x}$ и наоборот — противоречие с любым присваиванием.

Алгоритм через КСС

Критерий превращается в алгоритм: построить граф импликаций, найти компоненты сильной связности, проверить пары x, $\overline{x}$. Все шаги — стандартные линейные проходы по графу, без перебора присваиваний. Заодно алгоритм строит саму модель, а не только отвечает «да» или «нет».

- Поиск компонент — за $O(n + m)$ (Косарайю, Тарьян).
- Выполняющий набор строят по обратному топологическому порядку компонент: компоненте присваиваем 0, если она содержит отрицание уже выбранного литерала, иначе 1.

Граф импликаций



Пример

$(x \vee y) \wedge (\overline{x} \vee y)$.

Из $\overline{x}$ и из x — пути в y.

y и $\overline{y}$ в разных компонентах сильной связности, x и $\overline{x}$ — тоже: формула выполнима, $y = 1$.

Две стрелки дизъюнкта

Дизъюнкт порождает две стрелки: прямую и контрапозицию.

Потерять вторую — потерять ответ.

SAT и NP

> *Или математика слишком велика для человеческого разума, или человеческий разум больше, чем машина.*
>
> **— Курт Гёдель**

NP и сводимость

Определение 5

- **NP** — задачи, чей ответ «да» проверяется за полиномиальное время.
- **Полиномиальная сводимость** — преобразование входов, сохраняющее ответ.

Задача NP-полна, если она в NP и к ней сводится любая NP-задача.

Найти решение труднее, чем проверить — пока не доказано.

Пример

Судоку: заполненную сетку проверить легко.

Найти решение с нуля — перебор 9^{81}.

Теорема Кука

Теорема 2 (Кук, 1971)

SAT NP-полна.

Доказательство

SAT в NP: сертификат — выполняющий набор. Проверка за полиномиальное время.

NP-трудна: таблица вычисления машины Тьюринга кодируется в КНФ.

Формула выполнима тогда и только тогда, когда машина принимает вход.

Любая NP-задача сводится к SAT.

Свидетель становится моделью формулы.

P против NP

Следствие 1

Если SAT решается полиномиально, то $P = NP$.

Если SAT требует сверхполиномиального времени, то $P \neq NP$.

Вопрос «SAT в P?» эквивалентен вопросу «P = NP?».

Это открытый вопрос — одна из задач тысячелетия.

Если бы $P = NP$, распознать доказательство было бы так же легко, как найти.

Каждый, кто способен проследить рассуждение, был бы Гауссом.

Кодирование задач в SAT

> *Язык — орудие человеческого разума, а не просто средство выражения мысли.*
>
> **— Джордж Буль**

Стратегия

Определение 6 (Кодирование)

1. Выбрать булевы переменные, описывающие решение.
2. Записать ограничения дизъюнктами.
3. Запустить SAT-решатель.
4. Если решатель ответил SAT, декодируем модель в решение задачи, а если UNSAT — решения нет.

Хорошая кодировка наследует структуру задачи.

Плохая — раздувает число дизъюнктов экспоненциально.

Раскраска графа

Пример

Раскрасить граф в k цветов.

Переменные: $x_{v,c}$ — вершина v цвета c.

Ограничения:

- Каждая вершина имеет цвет: $\bigvee_{c=1}^{k} x_{v,c}$.
- Не более одного цвета: для всех $c < d$ — $\overline{x_{v,c}} \vee \overline{x_{v,d}}$.
- Смежные различаются: для каждого ребра (u, v) и цвета c — $\overline{x_{u,c}} \vee \overline{x_{v,c}}$.

«Хотя бы один» — один длинный дизъюнкт.

«Не более одного» — $\binom{k}{2}$ попарных запретов.

Судоку

Пример

Судоку 9×9 кодируется в КНФ напрямую.

$9^3 = 729$ переменных: $x_{r,c,d}$ — в клетке (r, c) стоит цифра d.

Ограничения:

- каждая клетка содержит ровно одну цифру;
- в каждой строке, столбце и блоке одна и та же цифра не встречается дважды.

Около 12 тысяч дизъюнктов — решатель щёлкает за миллисекунды.

N-ферзи

Пример (N-ферзи)

Переменные: $x_{i,j}$ — ферзь в клетке.

По одному в строке, столбце, на диагоналях.

При $N = 8$ — 736 дизъюнктов.

Решатель не знает правил судоку.

Ему дана бесструктурная КНФ — и он находит модель.

CDCL

Современные решатели — на алгоритме CDCL.

- **Unit propagation**: все литералы дизъюнкта ложны, кроме одного — он обязан быть истинным.
- **Решение**: эвристика VSIDS выбирает переменную.
- **Анализ конфликта**: извлекается выученный дизъюнкт.
- **Нехронологический возврат** + перезапуски.

- Классический перебор с возвратом (DPLL, Дэвис–Патнэм–Логеман–Лавленд) выбирает переменную, пробует оба её значения с unit-распространением и при конфликте возвращается, ничего не запоминая.
- CDCL запоминает их как выученные дизъюнкты.
- Это отсекает целые подпространства перебора.

Доказательство невыполнимости

На UNSAT решатель выдаёт разрешающее доказательство в резолюции, а не один ответ «нет»: независимая программа проверяет его за доли секунды.

Выученные дизъюнкты CDCL — это и есть шаги резолюции из лекции о логике.

Применения SAT

- **Верификация аппаратуры**: эквивалентность схем через митер (две схемы подаются на входы XOR, и формула невыполнима тогда и только тогда, когда схемы эквивалентны).
- **Ограниченная проверка моделей**: развёртывание программы в КНФ.
- **Криптоанализ**: поиск ключа для урезанного шифра.
- **Конфигурации**: зависимости пакетов Linux.

- NP-полная задача — систематически решаемая на индустриальных входах.
- Причина — структура индустриальных формул: переменные группируются в слабо связанные сообщества (граф формулы близок к дереву, мера близости — древесная ширина).
- У случайных формул структуры нет — там и экспонента.

Что мы поняли?

КНФ оказалась не ограничением, а интерфейсом: в неё кодируются раскраски, расписания, головоломки и схемы. Теория отвечает за границы: 2-SAT решается графом импликаций, 3-SAT NP-полна, теорема Кука связывает SAT со всем классом NP. Практика отвечает за скорость: CDCL запоминает конфликты и справляется с индустриальными формулами, на которых наивный перебор не заканчивается.

- SAT спрашивает, существует ли модель КНФ. Выполнимость не совпадает с общезначимостью.
- Сложность меняется скачком: 2-SAT решается через компоненты сильной связности графа импликаций, 3-SAT NP-полна.
- Сведение к SAT — переменные плюс ограничения. CDCL запоминает конфликты и отсекает перебор.
- Теорема Кука делает SAT эталоном NP: «SAT в P?» эквивалентно вопросу «P = NP?».

Вопросы для самопроверки

Вопросы собраны со всех разделов деки — от семантики КНФ до устройства CDCL. Ответ сильнее, если он построен руками: граф, кодировка, оценка размера формулы.

Вопросы для самопроверки — выполнимость и решатели

- Почему $(x) \wedge (\overline{x})$ невыполнима?
- Чем выполнимость отличается от общезначимости?
- Почему 2-SAT разрешима за полиномиальное время, а 3-SAT NP-полна?
- Постройте граф импликаций для $(x \vee y) \wedge (\overline{x} \vee \overline{y})$ и решите, выполнима ли формула.
- Закодируйте задачу «раскрасить треугольник в 2 цвета» в SAT.
- Что добавляет CDCL к DPLL?
- Почему SAT NP-полна и что из этого следует для P и NP?

Что читать дальше

Резолюцию, шаги которой запоминает CDCL, стоит освежить в деке о дедукции и системах вывода. Место SAT в карте сложности — классы P и NP, сводимость, иерархия классов — тема деки о сложности. В книге тема развёрнута в главе m14: там — устройство DPLL и CDCL, кодировки и приложения. Рядом — глава m30 о классах сложности и глава m02 о системах вывода.